

Міністерство освіти і науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»

Факультет інформатики та обчислювальної
техніки Кафедра обчислювальної техніки

Лабораторна робота №4

**з дисципліни «Об'єктно-орієнтоване
програмування»**

на тему

**«Рефакторинг та складні об'єкти з множинним
успадкуванням»**

Виконав:

Мащута Олександр
студент групи ІМ-051
номер у списку групи: 7

Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

Завдання

1. Виконати рефакторинг графічного редактора з виділенням класу `MyEditor` для керування об'єктами.
2. Реалізувати C#-аналог множинного успадкування поведінок базових класів для складених фігур.
3. Реалізувати складний графічний об'єкт «Лінія з кружечками» (`LineWithCirclesShape`).
4. Реалізувати складний графічний об'єкт «Каркас куба» (`CubeWireframeShape`).
5. Додати нові фігури до меню та розширити панель інструментів `ToolBar` до 6 кнопок.
6. Оформити звіт.

Завдання згідно варіанту

Для студента №7 (Ж = 7):

1. **Рефакторинг:** Клас `MyEditor` акумулює логіку зберігання (`List<Shape>`), очищення та малювання всіх створених об'єктів.

2. Множинне успадкування в C#: Оскільки C# не підтримує множинне успадкування класів, реалізовано C#-аналог за допомогою інтерфейсів із методами за замовчуванням (Default Interface Methods, C# 8+):

- `ILineBehavior` : визначає поведінку малювання відрізка лінії.
- `IEllipseBehavior` : визначає поведінку заповнення еліпса та кружечків.
- `IRectBehavior` : визначає поведінку заповнення прямокутника та контуру.

3. Складені фігури:

- `LineWithCirclesShape` : успадковує `Shape` та реалізує `ILineBehavior` + `IEllipseBehavior`.
- `CubewireframeShape` : успадковує `Shape` та реалізує `ILineBehavior` + `IRectBehavior`.

Вихідний текст програми

Клас керування об'єктами `MyEditor.cs`

```
using System;
using System.Collections.Generic;
using System.Drawing;
using Lab4.Shapes;

namespace Lab4
{
    public class MyEditor
    {
        private List<Shape> _shapes = new List<Shape>();

        public void AddShape(Shape shape)
        {
            _shapes.Add(shape);
        }

        public void DrawAll(Graphics g, Pen pen)
        {
            foreach (var shape in _shapes)
            {
                shape.Draw(g, pen, GetFillBrush(shape));
            }
        }

        private static Brush GetFillBrush(Shape shape) => shape switch
        {
            RectShape => Brushes.Orange,
            EllipseShape => Brushes.White,
            LineWithCirclesShape => Brushes.Yellow,
            CubewireframeShape => Brushes.Cyan,
```

```

        _ => Brushes.LightGray
    };

    public void Clear()
    {
        _shapes.Clear();
    }
}

```

Інтерфейси поведінки IShapeBehaviors.cs (C#-аналог множинного успадкування)

```

using System;
using System.Drawing;

namespace Lab4.Shapes
{
    public interface ILineBehavior
    {
        int X1 { get; } int Y1 { get; } int X2 { get; } int Y2 { get; }

        void DrawLine(Graphics g, Pen pen) => g.DrawLine(pen, X1, Y1, X2, Y2);

        void DrawLineSeg(Graphics g, Pen pen, int x1, int y1, int x2, int y2)
            => g.DrawLine(pen, x1, y1, x2, y2);
    }

    public interface IEllipseBehavior
    {
        int X1 { get; } int Y1 { get; } int X2 { get; } int Y2 { get; }

        void DrawEllipseFilled(Graphics g, Pen pen, Brush brush)
        {
            int x = Math.Min(X1, X2), y = Math.Min(Y1, Y2);
            int w = Math.Abs(X1 - X2), h = Math.Abs(Y1 - Y2);
            g.FillEllipse(brush, x, y, w, h);
            g.DrawEllipse(pen, x, y, w, h);
        }

        void DrawCircle(Graphics g, Brush brush, int cx, int cy, int r)
            => g.FillEllipse(brush, cx - r, cy - r, 2 * r, 2 * r);
    }

    public interface IRectBehavior
    {
        int X1 { get; } int Y1 { get; } int X2 { get; } int Y2 { get; }

        void DrawRectFilled(Graphics g, Pen pen, Brush brush)
        {
            int x = Math.Min(X1, X2), y = Math.Min(Y1, Y2);
            int w = Math.Abs(X1 - X2), h = Math.Abs(Y1 - Y2);
            g.FillRectangle(brush, x, y, w, h);
            g.DrawRectangle(pen, x, y, w, h);
        }

        void DrawRectOutline(Graphics g, Pen pen, int x, int y, int w, int h)
            => g.DrawRectangle(pen, x, y, w, h);
    }
}

```

Класи складених фігур CompositeShapes.cs

```
using System;
using System.Drawing;

namespace Lab4.Shapes
{
    // Еквівалент C++: class LineOOShape : public LineShape, public I
    public class LineWithCirclesShape : Shape, ILineBehavior, IEllip
    {
        public LineWithCirclesShape(int x1, int y1, int x2, int y2)

        public override void Draw(Graphics g, Pen pen, Brush brush)
        {
            ((ILineBehavior)this).DrawLine(g, pen);
            ((IEllipseBehavior)this).DrawCircle(g, brush, X1, Y1, 3);
            ((IEllipseBehavior)this).DrawCircle(g, brush, X2, Y2, 3);
        }
    }

    // Еквівалент C++: class CubeShape : public LineShape, public Rect
    public class CubeWireframeShape : Shape, ILineBehavior, IRectBeh
    {
        public CubeWireframeShape(int x1, int y1, int x2, int y2) : I

        public override void Draw(Graphics g, Pen pen, Brush brush)
        {
            int x = Math.Min(X1, X2), y = Math.Min(Y1, Y2);
            int w = Math.Abs(X1 - X2), h = Math.Abs(Y1 - Y2);
            int offset = w / 3;

            ((IRectBehavior)this).DrawRectOutline(g, pen, x, y, w, h);
            ((IRectBehavior)this).DrawRectOutline(g, pen, x + offset,

            ILineBehavior line = (ILineBehavior)this;
            line.DrawLineSeg(g, pen, x, y, x + offset, y + offset);
            line.DrawLineSeg(g, pen, x + w, y, x + w + offset, y + of
            line.DrawLineSeg(g, pen, x, y + h, x + offset, y + h + of
            line.DrawLineSeg(g, pen, x + w, y + h, x + w + offset, y

        }
    }
}
```

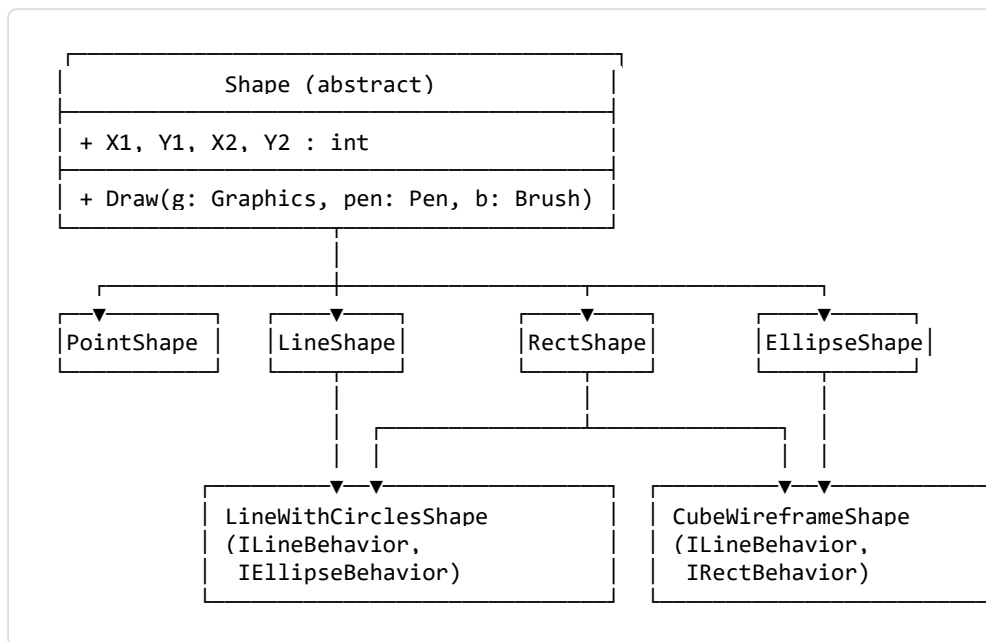
Діаграми

Структура файлів проєкту

```
Lab4
├─ Program.cs
├─ MainWindow.cs
├─ MvEditor.cs
├─ src/
│   ├── Shape.cs
│   ├── IShapeBehaviors.cs
│   ├── CompositeShapes.cs
│   ├── PointShape.cs
│   └── LineShape.cs
```

```
├─ RectShape.cs
├─ EllipseShape.cs
```

Діаграма класів (UML) із множинним успадкуванням



Скріншоти

Головне вікно з 6 кнопками Toolbar



Рис. 1. Оновлений Toolbar з 6 кнопками вибору інструментів

Малювання «Лінії з кружечками»



Рис. 2. Відображення складеної фігури LineWithCirclesShape

Малювання «Каркасу куба»



Висновки

У цій лабораторній роботі було здійснено рефакторинг графічного редактора з винесенням керування об'єктами в окремий клас `MyEditor`.

Також було реалізовано C#-аналог множинного успадкування за допомогою інтерфейсів з методами за замовчуванням (`Default Interface Methods`), що дозволило комбінувати поведінку декількох базових графічних елементів (ліній, еліпсів та прямокутників) для побудови складених об'єктів `LineWithCirclesShape` та `CubeWireframeShape`. Інтерфейс користувача оновлено до 6 інструментів.